CaseSifter

(formerly "Law Sifter")

Source Code Deposit -- Identifying Material

Submitted pursuant to 37 C.F.R. 202.20(c)(2)(vii) -- first and last portions of source code, trade-secret portions excluded.

Author/Claimant: John B. George
(c) 2026 John B. George. All rights reserved.

This excerpt omits internal modules containing the claimant's proprietary contradiction-detection and scoring logic.

```
===== FILE: backend/app/main.py =====

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app import cases_db
from app.api import router
from app.config import settings

app = FastAPI(
    title=settings.app_name,
    version=settings.app_version,
    description="Automated indicators of document alteration for court exhibit review.",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
app.include_router(router, prefix="/api")


@app.on_event("startup")
def _startup() -> None:
    cases_db.ensure_schema()


@app.get("/")
def root():
    return {
        "name": settings.app_name,
        "version": settings.app_version,
        "docs": "/docs",
        "health": "/api/health",
    }


===== FILE: backend/app/config.py =====

from pathlib import Path

from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=str(Path(__file__).resolve().parent.parent.parent / ".env"),
        extra="ignore"
    )

    app_name: str = "LawSifter"
    app_version: str = "1.0.0"
    max_file_mb: int = 100
    data_dir: Path = Path(__file__).resolve().parent.parent / "data"
    allowed_extensions: set[str] = {
        ".pdf",
        ".docx",
        ".xlsx",
        ".txt",
        ".jpg",
        ".jpeg",
        ".png",
        ".gif",
    }
    # Heavy image forensics is capped so a large exhibit cannot pin the machine.
    max_forensic_pages: int = 12
    render_scale: float = 1.15
    ela_quality: int = 90
```

```
    ela_scale: int = 18

    # Contextual engine, tier 2: claim verification. Off until a key is supplied.
    # Sends only generated questions to a third party, never document content.
    gemini_api_key: str | None = None
    gemini_model: str = "gemini-2.5-flash"
    gemini_timeout_s: int = 20
    contextual_max_chars: int = 40_000
    contextual_max_claims: int = 5
    contextual_min_confidence: int = 70
    contextual_window: int = 400

    # Contextual engine, tier 3: full document analysis. Requires a second, deliberate
    # opt-in because it uploads the original file and extracted text to Google.
    # Leave false for sealed, privileged, or otherwise restricted evidence.
    contextual_multimodal: bool = False
    # Inline request bodies are capped by the API; a larger exhibit degrades to text-only.
    contextual_max_upload_mb: int = 15
    contextual_max_ai_findings: int = 12
    gemini_document_timeout_s: int = 90
    # Cap Gemini "thinking" to cut latency and token burn. Gemini 3.x uses thinkingLevel
    # (minimal|low|medium|high). Gemini 2.5 Flash uses thinkingBudget (0 disables).
    gemini_thinking_level: str = "minimal"
    gemini_claim_thinking_level: str = "minimal"

    # Hybrid control plane. "open" unlocks scans locally; "license" needs LICENSE_KEY;
    # "cloud" expects login against CLOUD_CONTROL_URL (stubbed when URL is empty).
    entitlement_mode: str = "open"
    license_key: str | None = None
    cloud_control_url: str | None = None
    share_anonymized_outcomes: bool = True

    @property
    def jobs_dir(self) -> Path:
        return self.data_dir / "jobs"

    @property
    def cases_dir(self) -> Path:
        return self.data_dir / "cases"


settings = Settings()
settings.data_dir.mkdir(parents=True, exist_ok=True)
settings.jobs_dir.mkdir(parents=True, exist_ok=True)
settings.cases_dir.mkdir(parents=True, exist_ok=True)


===== FILE: backend/app/models.py =====

from __future__ import annotations

from datetime import datetime
from typing import Any, Literal

from pydantic import BaseModel, Field

Category = Literal[
    "metadata",
    "image",
    "text",
    "pdf_structure",
    "page_anomalies",
    "contextual",
    "case_consistency",
    "gotcha",
]
Severity = Literal["info", "low", "medium", "high", "critical"]
RiskLevel = Literal["green", "yellow", "red"]
JobStatus = Literal["queued", "running", "complete", "failed"]
```

```
class Finding(BaseModel):
    id: str
    category: Category
    severity: Severity
    title: str
    explanation: str
    why_suspicious: str
    confidence: float = Field(ge=0, le=100)
    page: int | None = None
    evidence: dict[str, Any] = Field(default_factory=dict)
    recommended_action: str | None = None


class TimelineEvent(BaseModel):
    label: str
    timestamp: str | None = None
    source: str
    note: str | None = None
    anomaly: bool = False


class PageDimension(BaseModel):
    page: int
    width_pts: float | None = None
    height_pts: float | None = None
    width_px: int | None = None
    height_px: int | None = None
    dpi: float | None = None
    rotation: int | None = None
    orientation: str | None = None


class FontUsage(BaseModel):
    name: str
    pages: list[int] = Field(default_factory=list)
    sizes: list[float] = Field(default_factory=list)
    sample: str | None = None


class ModuleResult(BaseModel):
    name: str
    status: Literal["ok", "skipped", "error"]
    duration_ms: int = 0
    summary: str = ""
    findings: list[Finding] = Field(default_factory=list)
    extras: dict[str, Any] = Field(default_factory=dict)


class AuditTrail(BaseModel):
    scanned_at: datetime
    analysis_version: str
    modules_run: list[str]
    submitted_by: str = "court-staff"
    filename: str
    file_size_bytes: int
    file_type: str
    sha256: str


class Report(BaseModel):
    job_id: str
    risk_level: RiskLevel
    risk_score: int = Field(ge=0, le=100)
    executive_summary: str
    top_findings: list[Finding] = Field(default_factory=list)
    recommended_actions: list[str] = Field(default_factory=list)
    findings: list[Finding] = Field(default_factory=list)
    timeline: list[TimelineEvent] = Field(default_factory=list)
    page_dimensions: list[PageDimension] = Field(default_factory=list)
```

```
    fonts: list[FontUsage] = Field(default_factory=list)
    comparison: dict[str, Any] = Field(default_factory=dict)
    modules: list[ModuleResult] = Field(default_factory=list)
    audit: AuditTrail
    artifact_urls: list[str] = Field(default_factory=list)
    disclaimer: str = (
        "This report identifies potential indicators of alteration or inconsistency. "
        "It is a decision-support tool, not a legal determination of authenticity, "
        "fraud, or admissibility. Findings should be reviewed by qualified personnel."
    )


class JobProgress(BaseModel):
    job_id: str
    status: JobStatus
    stage: str
    percent: int = Field(ge=0, le=100)
    message: str = ""
    filename: str
    error: str | None = None
    created_at: datetime
    updated_at: datetime
    report: Report | None = None


class UploadResponse(BaseModel):
    job_id: str
    filename: str
    status: JobStatus


class CaseCreate(BaseModel):
    caption: str = Field(min_length=1, max_length=300)
    court: str | None = None
    index_number: str | None = None
    notes: str | None = None


class CaseUpdate(BaseModel):
    caption: str | None = Field(default=None, min_length=1, max_length=300)
    court: str | None = None
    index_number: str | None = None
    notes: str | None = None


class DocumentUpdate(BaseModel):
    display_name: str | None = Field(default=None, min_length=1, max_length=200)
    doc_kind: str | None = None


class CaseChatBody(BaseModel):
    question: str = Field(min_length=1, max_length=2000)
    focus_document_id: str | None = None


class LicenseBody(BaseModel):
    license_key: str = Field(min_length=8, max_length=200)


class LoginBody(BaseModel):
    email: str
    password: str


class ShareOutcomesBody(BaseModel):
    enabled: bool


class FindingFeedbackBody(BaseModel):
    finding_id: str
```

```
    useful: bool
    category: str | None = None
    field: str | None = None
    doc_kind: str | None = None
    job_id: str | None = None


===== FILE: backend/app/storage.py =====

from __future__ import annotations

import json
import threading
from datetime import datetime, timezone
from pathlib import Path

from app.config import settings
from app.models import JobProgress, JobStatus, Report

_lock = threading.Lock()


def utcnow() -> datetime:
    return datetime.now(timezone.utc)


def job_dir(job_id: str) -> Path:
    path = settings.jobs_dir / job_id
    path.mkdir(parents=True, exist_ok=True)
    (path / "artifacts").mkdir(exist_ok=True)
    return path


def _progress_path(job_id: str) -> Path:
    return job_dir(job_id) / "progress.json"


def save_progress(progress: JobProgress) -> None:
    progress.updated_at = utcnow()
    path = _progress_path(progress.job_id)
    # Keep the report in report.json only. Embedding it here made every status
    # poll reread a multi-megabyte file and froze the UI after large exhibits.
    slim = progress.model_copy(update={"report": None})
    with _lock:
        path.write_text(slim.model_dump_json(indent=2), encoding="utf-8")


def load_progress(job_id: str) -> JobProgress | None:
    path = _progress_path(job_id)
    if not path.exists():
        return None
    return JobProgress.model_validate_json(path.read_text(encoding="utf-8"))


def create_job(job_id: str, filename: str) -> JobProgress:
    progress = JobProgress(
        job_id=job_id,
        status="queued",
        stage="queued",
        percent=0,
        message="File received. Analysis will begin shortly.",
        filename=filename,
        created_at=utcnow(),
        updated_at=utcnow(),
    )
    save_progress(progress)
    return progress


def update_job(
```

```
    job_id: str,
    *,
    status: JobStatus | None = None,
    stage: str | None = None,
    percent: int | None = None,
    message: str | None = None,
    error: str | None = None,
    report: Report | None = None,
) -> JobProgress:
    progress = load_progress(job_id)
    if progress is None:
        raise FileNotFoundError(f"Unknown job {job_id}")
    if status is not None:
        progress.status = status
    if stage is not None:
        progress.stage = stage
    if percent is not None:
        progress.percent = percent
    if message is not None:
        progress.message = message
    if error is not None:
        progress.error = error
    elif status in {"queued", "running"}:
        progress.error = None
    if report is not None:
        progress.report = report
        report_path = job_dir(job_id) / "report.json"
        report_path.write_text(report.model_dump_json(indent=2), encoding="utf-8")
    save_progress(progress)
    return progress


def load_report(job_id: str) -> Report | None:
    path = job_dir(job_id) / "report.json"
    if path.exists():
        return Report.model_validate_json(path.read_text(encoding="utf-8"))
    progress = load_progress(job_id)
    return progress.report if progress else None


def artifact_dir(job_id: str) -> Path:
    return job_dir(job_id) / "artifacts"


def save_json(path: Path, data: dict) -> None:
    path.write_text(json.dumps(data, indent=2, default=str), encoding="utf-8")


===== FILE: backend/app/telemetry.py =====

"""Anonymized finding telemetry for continuous evals — never sends exhibit binaries."""

from __future__ import annotations

import json
import urllib.error
import urllib.request
from typing import Any

from app import cases_db
from app.config import settings
from app.models import Finding


def share_enabled() -> bool:
    flag = cases_db.get_setting(
        "share_anonymized_outcomes",
        "true" if settings.share_anonymized_outcomes else "false",
    )
    return flag == "true"
```

```
def emit_scan_outcomes(
    *,
    doc_kind: str | None,
    findings: list[Finding],
    risk_level: str,
) -> str | None:
    if not share_enabled():
        return None
    payload = {
        "doc_kind": doc_kind or "unknown",
        "risk_level": risk_level,
        "findings": [
            {
                "category": f.category,
                "severity": f.severity,
                "confidence": round(float(f.confidence), 1),
                "analysis_type": (f.evidence or {}).get("analysis_type"),
                "case_consistency": bool((f.evidence or {}).get("case_consistency")),
                "field": (f.evidence or {}).get("field"),
            }
            for f in findings
            if f.severity != "info"
        ],
    }
    return cases_db.record_telemetry("scan_outcomes", payload)


def record_finding_feedback(
    *,
    finding_id: str,
    useful: bool,
    category: str | None = None,
    field: str | None = None,
    doc_kind: str | None = None,
) -> str:
    payload = {
        "finding_id_hash": finding_id[:24],
        "useful": useful,
        "category": category,
        "field": field,
        "doc_kind": doc_kind,
    }
    event_id = cases_db.record_telemetry("finding_feedback", payload)
    _try_flush()
    return event_id


def flush_to_cloud(limit: int = 50) -> dict[str, Any]:
    """Push unsynced anonymized events to the control plane when configured."""
    events = cases_db.list_unsynced_telemetry(limit=limit)
    if not events:
        return {"flushed": 0}
    if not settings.cloud_control_url:
        # Local-only: mark as synced so the queue does not grow forever in open mode.
        cases_db.mark_telemetry_synced([e["id"] for e in events])
        return {"flushed": len(events), "destination": "local"}

    token = cases_db.get_setting("cloud_session_token") or ""
    body = json.dumps(
        {
            "events": [
                {
                    "id": e["id"],
                    "type": e["event_type"],
                    "payload": e["payload"],
                    "created_at": e["created_at"],
                }
                for e in events
```

```
            ]
        }
    ).encode("utf-8")
    req = urllib.request.Request(
        settings.cloud_control_url.rstrip("/") + "/v1/telemetry",
        data=body,
        method="POST",
        headers={
            "Content-Type": "application/json",
            "Authorization": f"Bearer {token}",
        },
    )
    try:
        with urllib.request.urlopen(req, timeout=15) as resp:
            _ = resp.read()
        cases_db.mark_telemetry_synced([e["id"] for e in events])
        return {"flushed": len(events), "destination": "cloud"}
    except urllib.error.URLError as exc:
        return {"flushed": 0, "error": str(exc.reason)}


def _try_flush() -> None:
    try:
        flush_to_cloud(limit=20)
    except Exception:
        pass


===== FILE: backend/app/entitlement.py =====

"""Hybrid entitlement: local license / open mode, optional cloud control plane stub."""

from __future__ import annotations

import json
import urllib.error
import urllib.request
from typing import Any

from app import cases_db
from app.config import settings


def entitlement_status() -> dict[str, Any]:
    mode = settings.entitlement_mode
    share = cases_db.get_setting(
        "share_anonymized_outcomes",
        "true" if settings.share_anonymized_outcomes else "false",
    )
    base = {
        "mode": mode,
        "share_anonymized_outcomes": share == "true",
        "cloud_control_url": settings.cloud_control_url,
    }

    if mode == "open":
        return {
            **base,
            "active": True,
            "plan": "local-dev",
            "message": "Open mode — scans unlocked for local use.",
        }

    if mode == "license":
        key = (settings.license_key or cases_db.get_setting("license_key") or "").strip()
        active = bool(key) and len(key) >= 8
        return {
            **base,
            "active": active,
            "plan": "license" if active else None,
```

```
            "message": "License key accepted." if active else "Enter a valid license key to scan.",
            "license_present": bool(key),
        }

    token = cases_db.get_setting("cloud_session_token")
    if not token:
        return {
            **base,
            "active": False,
            "plan": None,
            "message": "Sign in to activate your monthly subscription.",
        }
    remote = _probe_cloud_entitlement(token)
    if remote is None:
        return {
            **base,
            "active": True,
            "plan": "cloud-cached",
            "message": "Using cached session; cloud control plane unreachable.",
            "cached": True,
        }
    return {**base, **remote}


def require_scan_allowed() -> dict[str, Any]:
    status = entitlement_status()
    if not status.get("active"):
        raise PermissionError(status.get("message") or "Subscription inactive.")
    return status


def save_license_key(key: str) -> dict[str, Any]:
    cases_db.set_setting("license_key", key.strip())
    return entitlement_status()


def cloud_login(email: str, password: str) -> dict[str, Any]:
    """Stub cloud login. Production hits the control plane; local mints a session."""
    email = email.strip().lower()
    if not email or "@" not in email or not password:
        raise ValueError("Email and password are required.")

    if settings.cloud_control_url:
        payload = json.dumps({"email": email, "password": password}).encode("utf-8")
        req = urllib.request.Request(
            settings.cloud_control_url.rstrip("/") + "/v1/login",
            data=payload,
            method="POST",
            headers={"Content-Type": "application/json"},
        )
        try:
            with urllib.request.urlopen(req, timeout=15) as resp:
                data = json.loads(resp.read().decode("utf-8"))
            token = data.get("token") or data.get("access_token")
            if not token:
                raise RuntimeError("Cloud login returned no token.")
            cases_db.set_setting("cloud_session_token", str(token))
            cases_db.set_setting("cloud_email", email)
            return entitlement_status()
        except urllib.error.URLError as exc:
            raise RuntimeError(f"Could not reach cloud control plane: {exc.reason}") from exc

    cases_db.set_setting("cloud_session_token", f"local-stub:{email}")
    cases_db.set_setting("cloud_email", email)
    return entitlement_status()


def cloud_logout() -> dict[str, Any]:
    cases_db.set_setting("cloud_session_token", "")
    return entitlement_status()
```

```
def set_share_outcomes(enabled: bool) -> dict[str, Any]:
    cases_db.set_setting("share_anonymized_outcomes", "true" if enabled else "false")
    return entitlement_status()


def _probe_cloud_entitlement(token: str) -> dict[str, Any] | None:
    if not settings.cloud_control_url:
        return {
            "active": True,
            "plan": "stub-monthly",
            "message": "Local cloud stub — subscription treated as active.",
            "email": cases_db.get_setting("cloud_email"),
        }
    req = urllib.request.Request(
        settings.cloud_control_url.rstrip("/") + "/v1/entitlement",
        method="GET",
        headers={"Authorization": f"Bearer {token}"},
    )
    try:
        with urllib.request.urlopen(req, timeout=10) as resp:
            data = json.loads(resp.read().decode("utf-8"))
        return {
            "active": bool(data.get("active")),
            "plan": data.get("plan"),
            "message": data.get("message") or "",
            "email": data.get("email") or cases_db.get_setting("cloud_email"),
        }
    except Exception:
        return None


===== FILE: backend/app/ingestion.py =====

from __future__ import annotations

import hashlib
import io
import zipfile
from pathlib import Path

from PIL import Image, ImageSequence

from app.config import settings

Image.MAX_IMAGE_PIXELS = 80_000_000

IMAGE_EXTS = {".jpg", ".jpeg", ".png", ".gif"}
RENDERABLE_EXTS = IMAGE_EXTS | {".pdf"}


def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()


def sniff_type(path: Path) -> str:
    suffix = path.suffix.lower()
    if suffix in settings.allowed_extensions:
        return suffix.lstrip(".")
    return "unknown"


def load_image(path: Path) -> Image.Image:
    image = Image.open(path)
    if path.suffix.lower() == ".gif":
```

```
        image = ImageSequence.Iterator(image)[0]
    if image.mode not in ("RGB", "L"):
        image = image.convert("RGB")
    elif image.mode == "L":
        image = image.convert("RGB")
    return image


def render_pdf_pages(path: Path, max_pages: int | None = None, scale: float | None = None):
    return list(iter_pdf_pages(path, max_pages=max_pages, scale=scale))


def iter_pdf_pages(path: Path, max_pages: int | None = None, scale: float | None = None):
    """Yield one rendered page at a time so large exhibits do not stay in RAM."""
    import pypdfium2 as pdfium

    scale = scale if scale is not None else settings.render_scale
    max_pages = max_pages if max_pages is not None else settings.max_forensic_pages
    doc = pdfium.PdfDocument(str(path))
    try:
        count = min(len(doc), max_pages)
        for index in range(count):
            page = doc[index]
            try:
                bitmap = page.render(scale=scale)
                yield bitmap.to_pil().convert("RGB")
            finally:
                page.close()
    finally:
        doc.close()


def extract_embedded_images(path: Path) -> list[tuple[str, Image.Image]]:
    """Pull raster images out of Office packages for forensic review."""
    suffix = path.suffix.lower()
    if suffix not in {".docx", ".xlsx"}:
        return []
    images: list[tuple[str, Image.Image]] = []
    media_prefix = "word/media/" if suffix == ".docx" else "xl/media/"
    try:
        with zipfile.ZipFile(path) as archive:
            for name in archive.namelist():
                if not name.startswith(media_prefix):
                    continue
                lower = name.lower()
                if not lower.endswith((".png", ".jpg", ".jpeg", ".gif", ".tif", ".tiff", ".bmp")):
                    continue
                raw = archive.read(name)
                try:
                    img = Image.open(io.BytesIO(raw)).convert("RGB")
                    images.append((name, img))
                except Exception:
                    continue
    except zipfile.BadZipFile:
        return []
    return images[:20]


def open_pdf_reader(path: Path):
    from pypdf import PdfReader

    reader = PdfReader(str(path), strict=False)
    if reader.is_encrypted:
        try:
            reader.decrypt("")
        except Exception:
            pass
    return reader
```

```
===== FILE: backend/app/scan_profile.py =====

"""Route documents to forensic vs court-filing scan profiles.

Exhibits (and unknown evidence packets) get the full alteration / whiteout / page-fraud
pipeline. Complaints, motions, and other pleadings are content + case-consistency only:
their packaging (AcroForm, mixed page size, e-filing stamps) is ordinary court workflow,
not exhibit forgery evidence.
"""

from __future__ import annotations

import re
from pathlib import Path

# Pleadings and procedural filings — extract facts, compare across the case.
COURT_FILING_KINDS = frozenset(
    {
        "complaint",
        "answer",
        "motion",
        "order",
        "affidavit",
        "affirmation",
        "letter",
        "stipulation",
        "transcript",
        "summons",
        "notice",
        "brief",
        "memorandum",
        "reply",
        "opposition",
        "filing",
        "judgment",
        "rji",
        "proof_of_service",
        "notice_of_pendency",
        "calendar",
    }
)

PLEADING_CAPTION_RE = re.compile(
    r"(?i)("
    r"supreme court of the state|"
    r"united states district court|"
    r"county court of the|"
    r"plaintiffs?\s*[,:]?\s*[-—]\s*against|"
    r"please take notice|"
    r"attorney\s+affirmation|"
    r"request for judicial intervention|"
    r"note of issue|"
    r"notice of pendency|"
    r"wherefore.{0,80}demands? judgment|"
    r"notice of appearance|"
    r"certificate of merit"
    r")"
)

STATEMENT_HINT_RE = re.compile(
    r"(?i)("
    r"account\s+(?:number|no\.?)|"
    r"amount\s+due|"
    r"statement\s+date|"
    r"previous\s+balance|"
    r"invoice\s+(?:number|no\.?)|"
    r"kwh|therms|"
    r"check\s+number"
    r")"
)
```

```
EFILING_HEADER_RE = re.compile(
    r"(?i)\b(?:NYSCEF|FILED:\s|INDEX\s+NO\.?|RECEIVED\s+NYSCEF)\b"
)

# Evidence packets — full fraudulent-document forensics.
EXHIBIT_KINDS = frozenset(
    {
        "exhibit",
        "bill",
        "statement",
        "invoice",
        "check",
        "receipt",
        "photo",
        "image",
        "evidence",
    }
)

PROFILE_FORENSIC = "forensic"
PROFILE_COURT_FILING = "court_filing"


def scan_profile_for_kind(doc_kind: str | None) -> str:
    kind = (doc_kind or "").strip().lower()
    if kind in COURT_FILING_KINDS:
        return PROFILE_COURT_FILING
    if kind in EXHIBIT_KINDS:
        return PROFILE_FORENSIC
    # Unknown is not a risk signal. Do not send administrative NYSCEF forms
    # through exhibit forensics just because the kind dropdown is blank.
    return PROFILE_COURT_FILING


def is_court_filing(doc_kind: str | None) -> bool:
    return scan_profile_for_kind(doc_kind) == PROFILE_COURT_FILING


def infer_profile_from_text(text: str) -> str | None:
    """When the user left kind blank, guess pleading vs exhibit from the first pages."""
    head = (text or "")[:8000]
    if not head.strip():
        return None
    looks_like_statement = bool(STATEMENT_HINT_RE.search(head))
    looks_like_pleading = bool(PLEADING_CAPTION_RE.search(head))
    if looks_like_statement and not looks_like_pleading:
        return PROFILE_FORENSIC
    if looks_like_pleading:
        return PROFILE_COURT_FILING
    if EFILING_HEADER_RE.search(head) and not looks_like_statement:
        return PROFILE_COURT_FILING
    return None


def resolve_scan_profile(doc_kind: str | None, peek_text: str = "") -> str:
    """Honor an explicit kind; otherwise infer from caption / e-filing furniture."""
    kind = (doc_kind or "").strip().lower()
    if kind in COURT_FILING_KINDS:
        return PROFILE_COURT_FILING
    if kind in EXHIBIT_KINDS:
        return PROFILE_FORENSIC
    inferred = infer_profile_from_text(peek_text)
    return inferred or PROFILE_COURT_FILING


def peek_pdf_text(path: Path, max_pages: int = 2) -> str:
    """Cheap first-page text for profile inference. Failures return empty."""
    if path.suffix.lower() != ".pdf":
        return ""
```

```
    try:
        from app.ingestion import open_pdf_reader

        reader = open_pdf_reader(path)
        chunks: list[str] = []
        for page in reader.pages[:max_pages]:
            chunks.append(page.extract_text() or "")
        return "\n".join(chunks)
    except Exception:
        return ""


===== FILE: backend/app/auto_title.py =====

"""Heuristics to propose a unique display name / document kind for an uploaded filing."""

from __future__ import annotations

import re
from pathlib import Path

# Filename / NYSCEF type first. Caption body almost always contains "complaint"
# or "answer" and must not override AFFIDAVIT OF SERVICE, NOTICE OF MOTION, etc.
FILENAME_KIND_PATTERNS: list[tuple[str, re.Pattern[str]]] = [
    ("proof_of_service",
    +-> re.compile(r"(?i)\b(affidavit|affirmation)\b.{0,40}\bof\s+service\b|\baos\b")),
    ("motion", re.compile(
r"(?i)\bnotice\s+of\s+(?:cross[\s-]*)?mot(?:ion)?\b|\bcross[\s-]mot(?:ion)?\b|\bmotion\s+to\b"
    )),
    ("rji", re.compile(r"(?i)\brji\b|request\s+for\s+judicial\s+intervention")),
    ("filing", re.compile(
        r"(?i)authorization\s+for\s+electronic|statement\s+of\s+author|"
        r"certificate\s+of\s+merit"
    )),
    ("notice_of_pendency", re.compile(r"(?i)\bnotice\s+of\s+pendency\b")),
    ("calendar", re.compile(r"(?i)\b(appearance\s+detail|court\s+appearance|calendar)\b")),
    ("notice", re.compile(r"(?i)\bnotice\s+of\s+(?:entry|appearance|rejection|cross|mortgage)\b")),
    ("order", re.compile(r"(?i)\b(proposed\s+)?order\b|decision\s+and\s+order")),
    ("affidavit", re.compile(r"(?i)\baffidavit\b")),
    ("affirmation", re.compile(r"(?i)\baffirmation\b")),
    ("letter", re.compile(r"(?i)\b(cover\s+)?letter\b")),
    ("stipulation", re.compile(r"(?i)\bstipulation\b")),
    ("transcript", re.compile(r"(?i)\b(transcript|deposition)\b")),
    ("bill", re.compile(r"(?i)\b(utility\s+bill|account\s+statement|bank\s+statement)\b")),
    ("summons", re.compile(r"(?i)\bsummons\b")),
    ("complaint", re.compile(r"(?i)\b(verified\s+)?complaint\b")),
    ("answer", re.compile(r"(?i)\banswer\b")),
]

# Body peek: specific types only. Never complaint/answer — those words live in every caption.
BODY_KIND_PATTERNS: list[tuple[str, re.Pattern[str]]] = [
    pair
    for pair in FILENAME_KIND_PATTERNS
    if pair[0]
    in {
        "proof_of_service",
        "motion",
        "affidavit",
        "affirmation",
        "order",
        "letter",
        "stipulation",
        "transcript",
        "bill",
    }
]

# Filename like EXHIBIT_S__83 or Exhibit S_41 — letter plus the docket/file number.
EXHIBIT_FILE_RE = re.compile(
    r"(?i)(?:^|[\s_\-])exhibit[\s_\-]*"
```

```
    r"([A-Za-z0-9]{1,4}(?:-[A-Za-z0-9]{1,3})?)"
    r"(?:[\s_\-]+(\d{1,4}))?"
    r"(?=$|[\s_\-.(])"
)


def propose_document_title(
    filename: str,
    text_sample: str = "",
) -> tuple[str, str | None]:
    """Return (display_name, doc_kind). Filename wins over body text."""
    stem = _stem(filename)
    exhibit = exhibit_title_from_filename(filename)
    if exhibit:
        return _truncate(exhibit), "exhibit"

    cleaned = re.sub(r"[_\-]+", " ", stem)
    cleaned = re.sub(r"\s+", " ", cleaned).strip()
    # Kind may come from the file body; the visible name stays filename-based
    # so two uploads cannot collapse to the same short label.
    kind = _kind_from_filename(cleaned) or _kind_from_body(text_sample[:4000])
    if kind in {"affidavit", "affirmation"} and _looks_like_service(text_sample):
        kind = "proof_of_service"
    if kind == "motion":
        motion = _motion_title(cleaned) or _motion_title(text_sample[:4000])
        if motion:
            return _truncate(_with_file_number(motion, stem)), "motion"
    if kind:
        label = {
            "complaint": "Complaint",
            "answer": "Answer",
            "order": "Order",
            "affidavit": "Affidavit",
            "affirmation": "Affirmation",
            "letter": "Letter",
            "stipulation": "Stipulation",
            "transcript": "Transcript",
            "bill": "Statement",
            "summons": "Summons",
            "proof_of_service": "Affidavit of Service",
            "filing": "Filing",
            "notice": "Notice",
            "rji": "RJI",
            "notice_of_pendency": "Notice of Pendency",
            "calendar": "Calendar",
        }.get(kind, kind.title())
        titled = _filename_title(cleaned)
        name = titled if _keeps_file_identity(titled, stem) else f"{label} — {titled}"
        return _truncate(_with_file_number(name, stem)), kind

    if re.search(r"\b(exhibit|complaint|motion|affidavit)\b", cleaned, re.I):
        return _truncate(_with_file_number(cleaned, stem)), None

    return _truncate(_with_file_number(cleaned or filename, stem)), None


def exhibit_title_from_filename(filename: str) -> str | None:
    stem = _stem(filename)
    match = EXHIBIT_FILE_RE.search(stem)
    if not match:
        return None
    tag = match.group(1).upper()
    number = match.group(2)
    if number:
        return f"Exhibit {tag}_{number}"
    return f"Exhibit {tag}"


def uniquify_display_name(
    name: str,
```

```
    existing: list[str] | set[str],
    filename: str = "",
) -> str:
    """Guarantee a case-unique display name. Prefer the filename's S_## token."""
    proposed = (name or "").strip() or exhibit_title_from_filename(filename) or _stem(filename) or
    +-> "Untitled document"
    exhibit = exhibit_title_from_filename(filename)
    if exhibit:
        proposed = exhibit
    taken = {_norm_name(item) for item in existing if item}
    if _norm_name(proposed) not in taken:
        return _truncate(proposed)
    numbered = _with_file_number(proposed, _stem(filename))
    if _norm_name(numbered) not in taken:
        return _truncate(numbered)
    suffix = 2
    while True:
        candidate = f"{proposed} ({suffix})"
        if _norm_name(candidate) not in taken:
            return _truncate(candidate)
        suffix += 1


def _stem(filename: str) -> str:
    stem = Path(filename or "").stem
    return re.sub(r"\s*\(\d+\)\s*$", "", stem).strip()


def _kind_from_filename(haystack: str) -> str | None:
    for kind, pattern in FILENAME_KIND_PATTERNS:
        if pattern.search(haystack or ""):
            return kind
    return None


def _kind_from_body(haystack: str) -> str | None:
    for kind, pattern in BODY_KIND_PATTERNS:
        if pattern.search(haystack or ""):
            return kind
    return None


def _looks_like_service(text: str) -> bool:
    return bool(
        re.search(
            r"(?i)\b("
            r"affidavit\s+of\s+service|proof\s+of\s+service|"
            r"served\s+the\s+within|i\s+served|process\s+server"
            r")\b",
            text or "",
        )
    )


def _kind_from_text(haystack: str) -> str | None:
    """Filename tokens first; body is only for specific types, never caption 'complaint'."""
    cleaned = re.sub(r"[_\-]+", " ", haystack or "")
    kind = _kind_from_filename(cleaned) or _kind_from_body(cleaned)
    if kind in {"affidavit", "affirmation"} and _looks_like_service(haystack):
        return "proof_of_service"
    return kind


def _motion_title(text: str) -> str | None:
    match = re.search(r"\bmotion\s+to\s+([a-z][a-z\s]{2,40})", text or "", re.I)
    if not match:
        return None
    detail = re.sub(r"\s+", " ", match.group(1)).strip(" .")
    return f"Motion to {detail.title()}" if detail else "Motion"
```

```
def _filename_title(cleaned: str) -> str:
    if cleaned.islower():
        return cleaned.title()
    return cleaned


def _keeps_file_identity(title: str, stem: str) -> bool:
    number = _trailing_number(stem)
    if number and re.search(rf"(?:^|[\s_]){re.escape(number)}(?:$|[\s_])", title):
        return True
    return len(title) >= 16


def _with_file_number(title: str, stem: str) -> str:
    number = _trailing_number(stem)
    if not number:
        return title
    if re.search(rf"(?:^|[\s_]|No\.?\s*){re.escape(number)}(?:$|[\s_.])", title):
        return title
    if title.lower().startswith("exhibit "):
        return f"{title}_{number}"
    return f"{title} {number}"


def _trailing_number(stem: str) -> str | None:
    match = re.search(r"(?:^|[\s_\-])(\d{1,4})$", stem or "")
    return match.group(1) if match else None


def _norm_name(value: str) -> str:
    return re.sub(r"\s+", " ", (value or "").strip().lower())


def retitle_case_documents(case_id: str) -> list[dict[str, str]]:
    """Rebuild display names from original filenames so no two files share a name."""
    from app import cases_db

    docs = cases_db.list_documents(case_id)
    snaps = {item["document_id"]: item for item in cases_db.list_case_snapshots(case_id)}
    taken: list[str] = []
    changes: list[dict[str, str]] = []
    for doc in docs:
        filename = doc.get("original_filename") or ""
        excerpt = (snaps.get(doc["id"]) or {}).get("text_excerpt") or ""
        proposed, kind = propose_document_title(filename, excerpt[:4000])
        unique = uniquify_display_name(proposed, taken, filename)
        taken.append(unique)
        fields: dict[str, str] = {}
        if unique != (doc.get("display_name") or ""):
            fields["display_name"] = unique
        current_kind = (doc.get("doc_kind") or "").strip().lower()
        if kind and kind != current_kind:
            fields["doc_kind"] = kind
        if fields:
            cases_db.update_document(doc["id"], **fields)
            changes.append(
                {
                    "id": doc["id"],
                    "before": doc.get("display_name") or "",
                    "after": unique,
                    "kind_before": current_kind,
                    "kind_after": kind or current_kind,
                }
            )
    return changes


def _truncate(value: str, limit: int = 120) -> str:
    value = value.strip() or "Untitled document"
```

```
    return value if len(value) <= limit else value[: limit - 1].rstrip() + "…"


===== FILE: backend/app/evidence_tier.py =====

"""Classify case documents as real evidence vs advocacy.

Gotcha Finder may only use a quote as the counter-statement when it is Tier 1
(sworn, documentary, or an entered court act). Two unsworn motion narratives
disagreeing is litigation, not a gotcha.
"""

from __future__ import annotations

import calendar
import re
from datetime import date
from typing import Any

TIER_EVIDENCE = 1
TIER_ADVOCACY = 2

TIER_LABELS = {
    TIER_EVIDENCE: "evidence",
    TIER_ADVOCACY: "advocacy",
}

EVIDENCE_KINDS = frozenset(
    {
        "exhibit",
        "bill",
        "statement",
        "invoice",
        "check",
        "receipt",
        "photo",
        "image",
        "evidence",
        "proof_of_service",
        "transcript",
        "judgment",
    }
)

ADVOCACY_KINDS = frozenset(
    {
        "motion",
        "complaint",
        "answer",
        "brief",
        "memorandum",
        "reply",
        "opposition",
        "letter",
        "notice",
        "affirmation",
        "rji",
        "calendar",
        "notice_of_pendency",
        "filing",
    }
)

PROPOSED_ACT = re.compile(
    r"(?i)\b("
    r"proposed\s+(?:order|judgment|decision|findings?)|"
    r"settle(?:ment)?\s+(?:this\s+)?order|"
    r"counter-?order|"
    r"please\s+take\s+notice\s+that.{0,80}proposed"
    r")\b"
```

```
)
ENTERED_ORDER = re.compile(
    r"(?i)\b("
    r"so-?ordered|"
    r"enter(?:ed|ing)\s+(?:this\s+)?(?:order|judgment)|"
    r"it\s+is\s+(?:hereby\s+)?ordered|"
    r"the\s+clerk\s+(?:is\s+)?directed|"
    r"e-?filed\s+order|"
    r"decision\s+and\s+order"
    r")\b"
)
SWORN_RECORD = re.compile(
    r"(?i)\b("
    r"affidavit\s+of\s+service|"
    r"sworn\s+(?:to|before)|"
    r"duly\s+sworn|"
    r"being\s+duly\s+sworn|"
    r"notary|"
    r"subscribed\s+and\s+sworn|"
    r"under\s+penalty\s+of\s+perjury|"
    r"business\s+records?|"
    r"certified\s+copy|"
    r"official\s+record"
    r")\b"
)
SERVICE_RECORD = re.compile(
    r"(?i)\baffidavit\s+of\s+service|proof\s+of\s+service|process\s+server|"
    r"served\s+the\s+within\b"
)
WEEKDAY_COMMA_DATE = re.compile(
    r"(?i)\b(Monday|Tuesday|Wednesday|Thursday|Friday|Saturday|Sunday)"
    r",?\s+(January|February|March|April|May|June|July|August|September|October|November|December)"
    r"\s+(\d{1,2}),?\s+(\d{4})\b"
)
DATE_A_WEEKDAY = re.compile(
    r"(?i)\b(January|February|March|April|May|June|July|August|September|October|November|December)"
    r"\s+(\d{1,2}),?\s+(\d{4}),?\s+(?:a\s+|on\s+)?(Monday|Tuesday|Wednesday|Thursday|Friday|Saturday
    +-> |Sunday)\b"
)

_MONTHS = {
    "january": 1,
    "february": 2,
    "march": 3,
    "april": 4,
    "may": 5,
    "june": 6,
    "july": 7,
    "august": 8,
    "september": 9,
    "october": 10,
    "november": 11,
    "december": 12,
}
_WEEKDAYS = {
    "monday": 0,
    "tuesday": 1,
    "wednesday": 2,
    "thursday": 3,
    "friday": 4,
    "saturday": 5,
    "sunday": 6,
}


def classify_tier(
    doc: dict[str, Any] | None = None,
    quote: str = "",
    source_name: str = "",
    doc_kind: str | None = None,
```

```
) -> int:
    """Return 1 (evidence) or 2 (advocacy). Ambiguous advocacy wins."""
    kind = (doc_kind or (doc or {}).get("doc_kind") or "").strip().lower()
    name = " ".join(
        [
            source_name,
            (doc or {}).get("display_name") or "",
            (doc or {}).get("original_filename") or "",
        ]
    )
    blob = f"{name}\n{quote}"

    if PROPOSED_ACT.search(blob) or re.search(r"(?i)\bproposed\b", name):
        return TIER_ADVOCACY
    if SERVICE_RECORD.search(name) or (kind == "affidavit" and SERVICE_RECORD.search(blob)):
        return TIER_EVIDENCE
    if kind in EVIDENCE_KINDS:
        return TIER_EVIDENCE
    if kind == "stipulation":
        return TIER_EVIDENCE
    if kind == "order":
        if PROPOSED_ACT.search(blob) or re.search(r"(?i)\bproposed\b", name):
            return TIER_ADVOCACY
        if ENTERED_ORDER.search(blob) or re.search(r"(?i)\b(decision\s+and\s+order|so-?ordered)\b",
    +-> name):
            return TIER_EVIDENCE
        return TIER_ADVOCACY
    if kind == "affidavit":
        return TIER_EVIDENCE
    if kind in ADVOCACY_KINDS:
        return TIER_ADVOCACY
    if re.search(r"(?i)\bexhibit\b", name):
        return TIER_EVIDENCE
    if re.search(r"(?i)\b(motion|memorandum|proposed\s+order|opposition|reply\s+affirmation)\b",
    +-> name):
        return TIER_ADVOCACY
    if SWORN_RECORD.search(blob) or re.search(
        r"(?i)\b(notary|executed|hospital|intake|register\s+video|phone\s+extract|"
        r"school\s+sign-?out|bank\s+letter|treating\s+note|stipulation|"
        r"affidavit|exhibit|email)\b",
        quote,
    ):
        return TIER_EVIDENCE
    return TIER_ADVOCACY


def source_looks_like_advocacy(name: str) -> bool:
    return bool(
        re.search(
            r"(?i)("
            r"^\s*(motion|complaint|answer|memorandum|opposition|reply|proposed)|"
            r"\b(motion\s+to|proposed\s+order|reargue|memorandum\s+of\s+law)\b"
            r")",
            name or "",
        )
    )


def evidence_is_probative(candidate: dict[str, Any], docs: list[dict[str, Any]] | None = None) ->
    +-> bool:
    """True only when the counter-quote is Tier 1 evidence."""
    if candidate.get("evidence_tier") == TIER_ADVOCACY:
        return False
    if candidate.get("evidence_tier") == TIER_EVIDENCE:
        return True
    source = str(candidate.get("evidence_source") or "")
    if source_looks_like_advocacy(source):
        return False
    doc = _lookup_doc(None, source, docs) or _lookup_doc(candidate.get("document_id"), source, docs)
    return (
```

```
        classify_tier(
            doc,
            str(candidate.get("evidence_quote") or ""),
            source,
            candidate.get("evidence_kind") or (doc or {}).get("doc_kind"),
        )
        == TIER_EVIDENCE
    )


def claim_tier(candidate: dict[str, Any], docs: list[dict[str, Any]] | None = None) -> int:
    if candidate.get("claim_tier") in {TIER_EVIDENCE, TIER_ADVOCACY}:
        return int(candidate["claim_tier"])
    doc = _lookup_doc(candidate.get("claim_document_id"), candidate.get("claim_source"), docs)
    return classify_tier(
        doc,
        str(candidate.get("claim_quote") or ""),
        str(candidate.get("claim_source") or ""),
        candidate.get("claim_kind"),
    )


def is_self_undercut(candidate: dict[str, Any]) -> bool:
    """Claim and counter-proof appear to come from the same side or the same file."""
    claim_id = candidate.get("claim_document_id")
    ev_id = candidate.get("document_id")
    if claim_id and ev_id and claim_id == ev_id:
        return True
    claim_src = str(candidate.get("claim_source") or "").lower()
    ev_src = str(candidate.get("evidence_source") or "").lower()
    if claim_src and ev_src and claim_src == ev_src:
        return True
    tag = re.search(r"(?i)\bexhibit\s+[\"']?([A-Z0-9]{1,4})\b", claim_src + " " +
    +-> str(candidate.get("claim_quote") or ""))
    if tag and re.search(rf"(?i)\bexhibit\s+[\"']?{re.escape(tag.group(1))}\b", ev_src):
        return True
    return False


def counter_label(tier: int) -> str:
    return "Evidence" if tier == TIER_EVIDENCE else "Opposing assertion"


def find_internal_evidence_defects(
    *,
    text: str,
    source: str,
    document_id: str,
    doc_kind: str | None = None,
) -> list[dict[str, Any]]:
    """Tier 1 documents only: dates that name the wrong weekday."""
    if classify_tier({"doc_kind": doc_kind, "display_name": source}, text, source, doc_kind) !=
    +-> TIER_EVIDENCE:
        return []
    out: list[dict[str, Any]] = []
    for stated, actual, quote in _weekday_conflicts(text):
        out.append(
            {
                "title": "Apparent chronology inconsistency",
                "claim_quote": quote,
                "claim_source": source,
                "claim_document_id": document_id,
                "claim_kind": doc_kind,
                "claim_tier": TIER_EVIDENCE,
                "evidence_quote": f"The calendar date in that sentence falls on {actual}, not
    +-> {stated}.",
                "evidence_source": source,
                "document_id": document_id,
                "evidence_kind": doc_kind,
                "evidence_tier": TIER_EVIDENCE,
```

```
                "why_it_may_matter": (
                    "A sworn or documentary record that names the wrong weekday for its own date "
                    "is internally inconsistent and worth verifying before relying on it."
                ),
                "self_undercut": True,
            }
        )
    return out[:3]


def _weekday_conflicts(text: str) -> list[tuple[str, str, str]]:
    found: list[tuple[str, str, str]] = []
    for match in WEEKDAY_COMMA_DATE.finditer(text or ""):
        stated, month, day, year = match.group(1), match.group(2), match.group(3), match.group(4)
        actual = _actual_weekday(month, day, year)
        if actual and actual != stated.lower():
            found.append((stated, actual.title(), match.group(0)))
    for match in DATE_A_WEEKDAY.finditer(text or ""):
        month, day, year, stated = match.group(1), match.group(2), match.group(3), match.group(4)
        actual = _actual_weekday(month, day, year)
        if actual and actual != stated.lower():
            found.append((stated, actual.title(), match.group(0)))
    return found


def _actual_weekday(month: str, day: str, year: str) -> str | None:
    try:
        parsed = date(int(year), _MONTHS[month.lower()], int(day))
    except (KeyError, ValueError):
        return None
    return calendar.day_name[parsed.weekday()].lower()


def _lookup_doc(
    document_id: str | None,
    name: str | None,
    docs: list[dict[str, Any]] | None,
) -> dict[str, Any] | None:
    if not docs:
        return None
    if document_id:
        for doc in docs:
            if doc.get("id") == document_id:
                return doc
    query = (name or "").strip().lower()
    if not query:
        return None
    for doc in docs:
        hay = f"{doc.get('display_name') or ''} {doc.get('original_filename') or
    +-> ''}".strip().lower()
        if hay == query or query in hay or hay in query:
            return doc
    return None


===== FILE: backend/app/engines/__init__.py =====

from __future__ import annotations

import time
from collections.abc import Callable
from typing import Any

from app.models import Finding, ModuleResult


def timed_module(name: str, fn: Callable[[], tuple[list[Finding], dict[str, Any]]]) -> ModuleResult:
    started = time.perf_counter()
    try:
        findings, extras = fn()
```

```
        duration = int((time.perf_counter() - started) * 1000)
        return ModuleResult(
            name=name,
            status="ok",
            duration_ms=duration,
            summary=f"{len(findings)} finding(s)",
            findings=findings,
            extras=extras,
        )
    except Exception as exc:
        duration = int((time.perf_counter() - started) * 1000)
        return ModuleResult(
            name=name,
            status="error",
            duration_ms=duration,
            summary=f"Module failed: {exc}",
            findings=[],
            extras={"error": str(exc)},
        )


def finding(
    *,
    fid: str,
    category: str,
    severity: str,
    title: str,
    explanation: str,
    why_suspicious: str,
    confidence: float,
    page: int | None = None,
    evidence: dict | None = None,
    recommended_action: str | None = None,
) -> Finding:
    return Finding(
        id=fid,
        category=category,  # type: ignore[arg-type]
        severity=severity,  # type: ignore[arg-type]
        title=title,
        explanation=explanation,
        why_suspicious=why_suspicious,
        confidence=round(min(100.0, max(0.0, confidence)), 1),
        page=page,
        evidence=evidence or {},
        recommended_action=recommended_action,
    )
```

[Remaining source files omitted -- contain claimant's proprietary detection, scoring, and consistency-analysis algorithms.]

```
===== FILE: backend/app/report/__init__.py =====

from app.report.generator import render_html, render_pdf
from app.report.scoring import executive_summary, pick_top_findings, recommended_actions, score_risk

__all__ = [
    "render_html",
    "render_pdf",
    "executive_summary",
    "pick_top_findings",
    "recommended_actions",
    "score_risk",
]


===== FILE: backend/app/report/scoring.py =====

from __future__ import annotations

from collections import Counter

from app.finding_policy import is_material_finding
from app.gotcha import TYPE_SCORE_WEIGHT
from app.models import Finding, Report, RiskLevel


SEVERITY_WEIGHT = {
    "info": 0,
    "low": 6,
    "medium": 18,
    "high": 36,
    "critical": 55,
}

SEVERITY_RANK = {"info": 0, "low": 1, "medium": 2, "high": 3, "critical": 4}


def score_risk(findings: list[Finding]) -> tuple[RiskLevel, int]:
    """Score from verified findings only — never document type or title.

    Two affidavits (or two exhibits) must score differently if one has verified
    gotchas and the other does not. Procedural and admission gotchas weigh more
    than a minor factual inconsistency.
    """
    actionable = [
        f
        for f in findings
        if f.severity != "info" and f.confidence >= 40 and is_material_finding(f)
    ]
    raw = 0
    for item in actionable:
        weight = SEVERITY_WEIGHT[item.severity] * (item.confidence / 100.0)
        if item.category == "gotcha":
            gotcha_type = str((item.evidence or {}).get("contradiction_type") or "")
            weight *= TYPE_SCORE_WEIGHT.get(gotcha_type, 1.15)
            if (item.evidence or {}).get("tainted_support"):
                weight *= 1.5
            elif (item.evidence or {}).get("self_undercut"):
                weight *= 1.2
            elif (item.evidence or {}).get("evidence_tier") == 1 and (item.evidence or
    +-> {}).get("claim_tier") == 1:
                weight *= 1.1
        raw += int(weight)
    score = min(100, raw)

    if any(f.severity == "critical" and f.confidence >= 70 for f in actionable):
        return "red", max(score, 80)
    if any(f.severity == "high" and f.confidence >= 70 for f in actionable):
        return "red", max(score, 70)
    if any(f.severity in {"medium", "high"} and f.confidence >= 55 for f in actionable):
```

```
        return "yellow", max(score, 35)
    return "green", score


def pick_top_findings(findings: list[Finding], limit: int = 3) -> list[Finding]:
    material = [f for f in findings if is_material_finding(f)]
    pool = material or [f for f in findings if f.severity != "info"]
    ranked = sorted(
        pool,
        key=lambda f: (
            1 if (f.evidence or {}).get("tainted_support") else 0,
            1 if (f.evidence or {}).get("analysis_type") == "date_inconsistency" else 0,
            SEVERITY_RANK[f.severity],
            f.confidence,
        ),
        reverse=True,
    )
    return ranked[:limit]


def recommended_actions(risk: RiskLevel, findings: list[Finding]) -> list[str]:
    actions: list[str] = []
    seen: set[str] = set()
    for item in pick_top_findings(findings, 8):
        if item.recommended_action and item.recommended_action not in seen:
            seen.add(item.recommended_action)
            actions.append(item.recommended_action)
    if risk == "red":
        actions.append(
            "Do not treat this exhibit as verified. Flag it for the assigned attorney or clerk and
    +-> keep this report with the case file."
        )
    elif risk == "yellow":
        actions.append(
            "Review the highlighted findings before relying on this exhibit. Request a native
    +-> original if the file appears exported or rebuilt."
        )
    else:
        actions.append(
            "No high-confidence manipulation indicators were found. This does not prove
    +-> authenticity; it means the automated checks did not raise a material alarm."
        )
    # Deduplicate while preserving order
    unique: list[str] = []
    for item in actions:
        if item not in unique:
            unique.append(item)
    return unique[:6]


def executive_summary(risk: RiskLevel, filename: str, findings: list[Finding]) -> str:
    counts = Counter(f.severity for f in findings if is_material_finding(f))
    top = pick_top_findings(findings)
    if risk == "green":
        return (
            f"Automated review of “{filename}” did not find strong signs of digital alteration. "
            "Timestamps, structure, and image-forensics checks are consistent with a routine file. "
            "Staff should still apply ordinary evidentiary judgment."
        )
    lead = top[0].title if top else "inconsistent internal signals"
    if risk == "red":
        return (
            f"“{filename}” is assessed as high risk. The leading concern is: {lead}. "
            f"The scan recorded {counts.get('critical', 0)} critical, {counts.get('high', 0)} high,
    +-> "
            f"and {counts.get('medium', 0)} medium finding(s). Treat the exhibit as unverified until
    +-> reviewed."
        )
    return (
        f"“{filename}” is assessed as moderate risk. The leading concern is: {lead}. "
```

```
        "One or more checks found inconsistencies that deserve human review but are not, by
    +-> themselves, "
        "proof of fraud."
    )


===== FILE: backend/app/report/generator.py =====

from __future__ import annotations

import base64
from datetime import datetime, timezone
from pathlib import Path

from jinja2 import Environment, FileSystemLoader, select_autoescape
from reportlab.lib import colors
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import inch
from reportlab.platypus import (
    Image as RLImage,
    ListFlowable,
    ListItem,
    Paragraph,
    SimpleDocTemplate,
    Spacer,
    Table,
    TableStyle,
)

from app import __version__
from app.models import Report
from app.storage import artifact_dir, job_dir

TEMPLATE_DIR = Path(__file__).resolve().parent / "templates"
env = Environment(
    loader=FileSystemLoader(str(TEMPLATE_DIR)),
    autoescape=select_autoescape(["html"]),
)

RISK_LABEL = {"green": "Low risk", "yellow": "Moderate risk", "red": "High risk"}
CATEGORY_LABEL = {
    "metadata": "Metadata",
    "image": "Image manipulation",
    "text": "Text / OCR",
    "pdf_structure": "PDF structure",
    "page_anomalies": "Page anomalies",
    "contextual": "Contextual logic",
    "case_consistency": "Case consistency",
}


def render_html(report: Report) -> str:
    template = env.get_template("report.html")
    return template.render(
        report=_embed_heatmaps(report),
        risk_label=RISK_LABEL.get(report.risk_level, report.risk_level),
        category_label=CATEGORY_LABEL,
        generated=datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M UTC"),
        version=__version__,
    )


def _embed_heatmaps(report: Report) -> Report:
    clone = report.model_copy(deep=True)
    for collection in (clone.findings, clone.top_findings):
        for item in collection:
            heat = (item.evidence or {}).get("heatmap")
            if not heat:
                continue
```

```
            name = str(heat).rsplit("/", 1)[-1]
            path = artifact_dir(report.job_id) / name
            if path.exists():
                encoded = base64.b64encode(path.read_bytes()).decode("ascii")
                item.evidence["heatmap"] = f"{encoded}"
    return clone


def render_pdf(report: Report) -> Path:
    out = job_dir(report.job_id) / "report.pdf"
    styles = getSampleStyleSheet()
    styles.add(
        ParagraphStyle(
            name="CoverTitle",
            fontName="Times-Bold",
            fontSize=18,
            leading=22,
            textColor=colors.HexColor("#1a365d"),
            spaceAfter=6,
        )
    )
    styles.add(
        ParagraphStyle(
            name="Section",
            fontName="Times-Bold",
            fontSize=13,
            leading=16,
            textColor=colors.HexColor("#1a365d"),
            spaceBefore=14,
            spaceAfter=6,
        )
    )
    styles.add(
        ParagraphStyle(
            name="Body",
            fontName="Times-Roman",
            fontSize=10,
            leading=13,
            textColor=colors.HexColor("#1a202c"),
        )
    )
    styles.add(
        ParagraphStyle(
            name="Small",
            fontName="Times-Roman",
            fontSize=8,
            leading=11,
            textColor=colors.HexColor("#4a5568"),
        )
    )
    styles.add(
        ParagraphStyle(
            name="FindingTitle",
            fontName="Times-Bold",
            fontSize=11,
            leading=14,
            textColor=colors.HexColor("#1a365d"),
            spaceBefore=8,
        )
    )

    risk_color = {
        "green": colors.HexColor("#276749"),
        "yellow": colors.HexColor("#b7791f"),
        "red": colors.HexColor("#c53030"),
    }[report.risk_level]

    story = []
    story.append(Paragraph("LawSifter — Authenticity Report", styles["CoverTitle"]))
    story.append(Paragraph(report.audit.filename, styles["Body"]))
```

```
    story.append(Spacer(1, 8))
    badge = Table(
        [[Paragraph(f"<b>{RISK_LABEL[report.risk_level]}</b>  (score {report.risk_score}/100)",
    +-> styles["Body"])]],
        colWidths=[6.5 * inch],
    )
    badge.setStyle(
        TableStyle(
            [
                ("BACKGROUND", (0, 0), (-1, -1), colors.HexColor("#edf2f7")),
                ("TEXTCOLOR", (0, 0), (-1, -1), risk_color),
                ("LEFTPADDING", (0, 0), (-1, -1), 10),
                ("TOPPADDING", (0, 0), (-1, -1), 8),
                ("BOTTOMPADDING", (0, 0), (-1, -1), 8),
            ]
        )
    )
    story.append(badge)
    story.append(Spacer(1, 10))
    story.append(Paragraph("Executive summary", styles["Section"]))
    story.append(Paragraph(report.executive_summary, styles["Body"]))

    story.append(Paragraph("Recommended actions", styles["Section"]))
    story.append(
        ListFlowable(
            [ListItem(Paragraph(a, styles["Body"])) for a in report.recommended_actions],
            bulletType="bullet",
        )
    )

    story.append(Paragraph("Detailed findings", styles["Section"]))
    if not report.findings:
        story.append(Paragraph("No automated findings were recorded.", styles["Body"]))
    for item in report.findings:
        if item.severity == "info":
            continue
        cat = CATEGORY_LABEL.get(item.category, item.category)
        page = f" — page {item.page}" if item.page else ""
        story.append(
            Paragraph(
                f"{item.title}{page}  ({cat}, {item.severity}, {item.confidence:.0f}% confidence)",
                styles["FindingTitle"],
            )
        )
        story.append(Paragraph(item.explanation, styles["Body"]))
        story.append(Paragraph(f"<i>Why this is flagged:</i> {item.why_suspicious}",
    +-> styles["Small"]))
        heatmap = (item.evidence or {}).get("heatmap")
        if heatmap:
            name = str(heatmap).rsplit("/", 1)[-1]
            img_path = artifact_dir(report.job_id) / name
            if img_path.exists():
                story.append(Spacer(1, 4))
                try:
                    from PIL import Image as PILImage

                    with PILImage.open(img_path) as im:
                        aspect = im.height / max(im.width, 1)
                    width = 5.8 * inch
                    height = min(4.2 * inch, width * aspect)
                    story.append(RLImage(str(img_path), width=width, height=height))
                except Exception:
                    pass

    if report.timeline:
        story.append(Paragraph("Timeline", styles["Section"]))
        for event in report.timeline:
            mark = " ■" if event.anomaly else ""
            story.append(
                Paragraph(
```

```
                    f"<b>{event.label}</b>{mark} — {event.timestamp or 'unknown'} ({event.source})",
                    styles["Body"],
                )
            )
            if event.note:
                story.append(Paragraph(event.note, styles["Small"]))

    if report.page_dimensions:
        story.append(Paragraph("Page dimensions", styles["Section"]))
        header = ["Page", "W (pt)", "H (pt)", "W (px)", "H (px)", "DPI", "Orient."]
        rows = [header]
        for dim in report.page_dimensions:
            rows.append(
                [
                    str(dim.page),
                    _fmt(dim.width_pts),
                    _fmt(dim.height_pts),
                    _fmt(dim.width_px),
                    _fmt(dim.height_px),
                    _fmt(dim.dpi),
                    dim.orientation or "",
                ]
            )
        table = Table(rows, colWidths=[0.7 * inch, 0.9 * inch, 0.9 * inch, 0.9 * inch, 0.9 * inch,
    +-> 0.7 * inch, 0.9 * inch])
        table.setStyle(
            TableStyle(
                [
                    ("FONTNAME", (0, 0), (-1, 0), "Times-Bold"),
                    ("FONTSIZE", (0, 0), (-1, -1), 8),
                    ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#edf2f7")),
                    ("GRID", (0, 0), (-1, -1), 0.3, colors.HexColor("#cbd5e0")),
                    ("LEFTPADDING", (0, 0), (-1, -1), 4),
                    ("RIGHTPADDING", (0, 0), (-1, -1), 4),
                ]
            )
        )
        story.append(table)

    story.append(Paragraph("Audit trail", styles["Section"]))
    audit_lines = [
        f"Scanned at: {report.audit.scanned_at.isoformat()}",
        f"Submitted by: {report.audit.submitted_by}",
        f"Analysis version: {report.audit.analysis_version}",
        f"Modules: {', '.join(report.audit.modules_run)}",
        f"File type: {report.audit.file_type} ({report.audit.file_size_bytes} bytes)",
        f"SHA-256: {report.audit.sha256}",
    ]
    for line in audit_lines:
        story.append(Paragraph(line, styles["Small"]))
    story.append(Spacer(1, 12))
    story.append(Paragraph(report.disclaimer, styles["Small"]))

    doc = SimpleDocTemplate(
        str(out),
        pagesize=letter,
        title=f"Exhibit scan — {report.audit.filename}",
        leftMargin=0.75 * inch,
        rightMargin=0.75 * inch,
        topMargin=0.7 * inch,
        bottomMargin=0.7 * inch,
    )
    doc.build(story)
    return out


def _fmt(value) -> str:
    if value is None:
        return "—"
    if isinstance(value, float):
```

```
        return f"{value:.1f}"
    return str(value)


===== FILE: backend/app/report/templates/report.html =====

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>LawSifter report — {{ report.audit.filename }}</title>
  <style>
    :root { --navy:#1a365d; --ink:#1a202c; --muted:#4a5568; --line:#e2e8f0; --paper:#f7f4ef; }
    body { font-family: "Times New Roman", Times, serif; color: var(--ink); margin: 0; background:
    +-> white; }
    .wrap { max-width: 860px; margin: 0 auto; padding: 32px 28px 64px; }
    h1 { font-size: 26px; color: var(--navy); margin: 0 0 4px; }
    h2 { font-size: 18px; color: var(--navy); margin: 28px 0 10px; border-bottom: 1px solid
    +-> var(--line); padding-bottom: 4px; }
    .meta { color: var(--muted); font-size: 13px; margin-bottom: 16px; }
    .badge { display: inline-block; padding: 6px 12px; font-weight: bold; letter-spacing: .02em; }
    .green { background: #c6f6d5; color: #22543d; }
    .yellow { background: #fefcbf; color: #744210; }
    .red { background: #fed7d7; color: #742a2a; }
    .finding { border: 1px solid var(--line); padding: 12px 14px; margin: 10px 0; }
    .finding h3 { margin: 0 0 6px; font-size: 15px; color: var(--navy); }
    .why { color: var(--muted); font-style: italic; }
    img.evidence { max-width: 100%; border: 1px solid var(--line); margin-top: 8px; }
    table { border-collapse: collapse; width: 100%; font-size: 13px; }
    th, td { border: 1px solid var(--line); padding: 5px 8px; text-align: left; }
    th { background: #edf2f7; }
    .disclaimer { font-size: 12px; color: var(--muted); margin-top: 28px; }
    ul { margin-top: 6px; }
  </style>
</head>
<body>
  <div class="wrap">
    <h1>LawSifter</h1>
    <div class="meta">Authenticity verification report · {{ generated }} · software v{{ version
    +-> }}</div>
    <p><strong>File:</strong> {{ report.audit.filename }}  |  <strong>SHA-256:</strong> {{
    +-> report.audit.sha256 }}</p>
    <p>
      <span class="badge {{ report.risk_level }}">{{ risk_label }} ({{ report.risk_score
    +-> }}/100)</span>
    </p>

    <h2>Executive summary</h2>
    <p>{{ report.executive_summary }}</p>

    <h2>Recommended actions</h2>
    <ul>
      {% for action in report.recommended_actions %}
      <li>{{ action }}</li>
      {% endfor %}
    </ul>

    <h2>Top findings</h2>
    {% if report.top_findings %}
      {% for item in report.top_findings %}
      <div class="finding">
        <h3>{{ item.title }} <small>({{ category_label.get(item.category, item.category) }}, {{
    +-> item.confidence|round(0) }}%)</small></h3>
        <p>{{ item.explanation }}</p>
        <p class="why">{{ item.why_suspicious }}</p>
      </div>
      {% endfor %}
    {% else %}
      <p>No material findings.</p>
    {% endif %}
```

```
    <h2>Detailed findings</h2>
    {% for item in report.findings if item.severity != "info" %}
    <div class="finding">
      <h3>{{ item.title }}</h3>
      <p>
        {{ category_label.get(item.category, item.category) }}
        · {{ item.severity }}
        · {{ item.confidence|round(0) }}% confidence
        {% if item.page %}· page {{ item.page }}{% endif %}
      </p>
      <p>{{ item.explanation }}</p>
      <p class="why">Why this is flagged: {{ item.why_suspicious }}</p>
      {% if item.evidence and item.evidence.heatmap %}
      <p><img class="evidence" src="{{ item.evidence.heatmap }}" alt="Forensic heatmap" /></p>
      {% endif %}
      {% if item.evidence and item.evidence.ai_generated %}
      <p style="font-size: 12px; color: #999; border-left: 3px solid #ffb81c; padding-left: 8px;
    +-> margin: 8px 0;">
        <strong>■■ AI-Assisted Research Lead:</strong> This finding is based on an automated public-
    +-> knowledge check, not definitive evidence. Verify independently before relying on it.
      </p>
      {% endif %}
    </div>
    {% else %}
    <p>No automated findings were recorded.</p>
    {% endfor %}

    <h2>Timeline</h2>
    {% if report.timeline %}
    <table>
      <thead><tr><th>Event</th><th>Timestamp</th><th>Source</th><th>Note</th></tr></thead>
      <tbody>
        {% for event in report.timeline %}
        <tr>
          <td>{{ event.label }}{% if event.anomaly %} ■{% endif %}</td>
          <td>{{ event.timestamp or "—" }}</td>
          <td>{{ event.source }}</td>
          <td>{{ event.note or "" }}</td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
    {% else %}
    <p>No timeline events were extracted.</p>
    {% endif %}

    <h2>Page dimensions</h2>
    {% if report.page_dimensions %}
    <table>
      <thead>
        <tr><th>Page</th><th>W (pt)</th><th>H (pt)</th><th>W (px)</th><th>H
    +-> (px)</th><th>DPI</th><th>Orientation</th></tr>
      </thead>
      <tbody>
        {% for dim in report.page_dimensions %}
        <tr>
          <td>{{ dim.page }}</td>
          <td>{{ dim.width_pts or "—" }}</td>
          <td>{{ dim.height_pts or "—" }}</td>
          <td>{{ dim.width_px or "—" }}</td>
          <td>{{ dim.height_px or "—" }}</td>
          <td>{{ dim.dpi or "—" }}</td>
          <td>{{ dim.orientation or "—" }}</td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
    {% else %}
    <p>Not applicable for this file type.</p>
```

```
    {% endif %}

    <h2>Fonts</h2>
    {% if report.fonts %}
    <table>
      <thead><tr><th>Font</th><th>Pages</th><th>Sizes</th><th>Sample</th></tr></thead>
      <tbody>
        {% for font in report.fonts %}
        <tr>
          <td>{{ font.name }}</td>
          <td>{{ font.pages|join(", ") }}</td>
          <td>{{ font.sizes|join(", ") }}</td>
          <td>{{ font.sample or "" }}</td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
    {% else %}
    <p>No embedded font inventory (typical for images and scans).</p>
    {% endif %}

    <h2>Audit trail</h2>
    <table>
      <tr><th>Scanned at</th><td>{{ report.audit.scanned_at }}</td></tr>
      <tr><th>Submitted by</th><td>{{ report.audit.submitted_by }}</td></tr>
      <tr><th>Version</th><td>{{ report.audit.analysis_version }}</td></tr>
      <tr><th>Modules</th><td>{{ report.audit.modules_run|join(", ") }}</td></tr>
      <tr><th>Size</th><td>{{ report.audit.file_size_bytes }} bytes</td></tr>
    </table>

    <p class="disclaimer">{{ report.disclaimer }}</p>
  </div>
</body>
</html>


===== FILE: frontend/src/main.jsx =====

import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);


===== FILE: frontend/src/api.js =====

const jsonHeaders = { Accept: "application/json" };

async function safeDetail(res) {
  try {
    const data = await res.json();
    return data.detail || JSON.stringify(data);
  } catch {
    return res.statusText;
  }
}

async function readJson(res) {
  if (!res.ok) {
    throw new Error((await safeDetail(res)) || "Request failed.");
  }
  return res.json();
}
```

```
export async function uploadExhibit(file, submittedBy = "court-staff") {
  const body = new FormData();
  body.append("file", file);
  body.append("submitted_by", submittedBy);
  const res = await fetch("/api/upload", { method: "POST", body });
  return readJson(res);
}

export async function getJob(jobId, { includeReport = false } = {}) {
  const qs = includeReport ? "?include_report=true" : "";
  const res = await fetch(`/api/jobs/${jobId}${qs}`, { headers: jsonHeaders });
  return readJson(res);
}

export function exportUrl(jobId, kind) {
  return `/api/jobs/${jobId}/export/${kind}`;
}

export function documentFileUrl(caseId, documentId) {
  return `/api/cases/${caseId}/documents/${documentId}/file`;
}

export function jobSourceUrl(jobId) {
  return `/api/jobs/${jobId}/source`;
}

export async function listCases() {
  const res = await fetch("/api/cases", { headers: jsonHeaders });
  const data = await readJson(res);
  return data.cases || [];
}

export async function createCase(payload) {
  const res = await fetch("/api/cases", {
    method: "POST",
    headers: { ...jsonHeaders, "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });
  return readJson(res);
}

export async function getCase(caseId) {
  const res = await fetch(`/api/cases/${caseId}`, { headers: jsonHeaders });
  return readJson(res);
}

export async function archiveCase(caseId) {
  const res = await fetch(`/api/cases/${caseId}/archive`, { method: "POST", headers: jsonHeaders });
  return readJson(res);
}

export async function unarchiveCase(caseId) {
  const res = await fetch(`/api/cases/${caseId}/unarchive`, { method: "POST", headers: jsonHeaders
    +-> });
  return readJson(res);
}

export async function deleteCase(caseId) {
  const res = await fetch(`/api/cases/${caseId}`, { method: "DELETE", headers: jsonHeaders });
  return readJson(res);
}

export async function updateCase(caseId, payload) {
  const res = await fetch(`/api/cases/${caseId}`, {
    method: "PATCH",
    headers: { ...jsonHeaders, "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });
  return readJson(res);
}
```

```
function isTransientUploadError(err) {
  const text = String(err?.message || err || "");
  return /failed to fetch|networkerror|econnreset|econnrefused|502|503|504|database is
    +-> locked/i.test(text);
}

export async function uploadCaseDocument(caseId, file, { displayName, autoScan = true } = {}) {
  let lastError = new Error("Upload failed.");
  for (let attempt = 0; attempt < 3; attempt += 1) {
    const body = new FormData();
    body.append("file", file);
    if (displayName) body.append("display_name", displayName);
    body.append("auto_scan", autoScan ? "true" : "false");
    try {
      const res = await fetch(`/api/cases/${caseId}/documents`, { method: "POST", body });
      return await readJson(res);
    } catch (err) {
      lastError = err;
      if (!isTransientUploadError(err) || attempt === 2) break;
      await new Promise((resolve) => setTimeout(resolve, 600 * (attempt + 1)));
    }
  }
  throw lastError;
}

export async function patchDocument(caseId, documentId, payload) {
  const res = await fetch(`/api/cases/${caseId}/documents/${documentId}`, {
    method: "PATCH",
    headers: { ...jsonHeaders, "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });
  return readJson(res);
}

export async function listCaseGotchas(caseId) {
  const res = await fetch(`/api/cases/${caseId}/gotchas`, { headers: jsonHeaders });
  const data = await readJson(res);
  return {
    gotchas: data.gotchas || [],
    factConflicts: data.fact_conflicts || [],
  };
}

export async function listCaseChat(caseId) {
  const res = await fetch(`/api/cases/${caseId}/chat`, { headers: jsonHeaders });
  const data = await readJson(res);
  return data.messages || [];
}

export async function askCaseChat(caseId, question, { focusDocumentId } = {}) {
  const res = await fetch(`/api/cases/${caseId}/chat`, {
    method: "POST",
    headers: { ...jsonHeaders, "Content-Type": "application/json" },
    body: JSON.stringify({
      question,
      focus_document_id: focusDocumentId || undefined,
    }),
  });
  return readJson(res);
}

export async function scanDocument(caseId, documentId) {
  const res = await fetch(`/api/cases/${caseId}/documents/${documentId}/scan`, {
    method: "POST",
    headers: jsonHeaders,
  });
  return readJson(res);
}
```

```
export async function rescanCase(caseId) {
  const res = await fetch(`/api/cases/${caseId}/rescan`, {
    method: "POST",
    headers: jsonHeaders,
  });
  return readJson(res);
}

export async function getEntitlement() {
  const res = await fetch("/api/entitlement", { headers: jsonHeaders });
  return readJson(res);
}

export async function loginCloud(email, password) {
  const res = await fetch("/api/auth/login", {
    method: "POST",
    headers: { ...jsonHeaders, "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });
  return readJson(res);
}

export async function logoutCloud() {
  const res = await fetch("/api/auth/logout", { method: "POST", headers: jsonHeaders });
  return readJson(res);
}

export async function saveLicense(licenseKey) {
  const res = await fetch("/api/entitlement/license", {
    method: "POST",
    headers: { ...jsonHeaders, "Content-Type": "application/json" },
    body: JSON.stringify({ license_key: licenseKey }),
  });
  return readJson(res);
}

export async function setShareOutcomes(enabled) {
  const res = await fetch("/api/settings/share-outcomes", {
    method: "POST",
    headers: { ...jsonHeaders, "Content-Type": "application/json" },
    body: JSON.stringify({ enabled }),
  });
  return readJson(res);
}

export async function sendFindingFeedback(payload) {
  const res = await fetch("/api/telemetry/finding-feedback", {
    method: "POST",
    headers: { ...jsonHeaders, "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });
  return readJson(res);
}


===== FILE: frontend/src/index.css =====

:root {
  --navy: #1a2f4d;
  --navy-deep: #101c30;
  --gold: #a07a32;
  --paper: #f6f3ee;
  --paper-2: #efeae2;
  --card: #fffdf9;
  --ink: #1a1f28;
  --muted: #66707d;
  --line: #e4ddd2;
  --line-strong: #d3cbbd;
  --green: #2f6b4f;
  --green-bg: #e5f4ea;
```

```
  --yellow: #8a6a1f;
  --yellow-bg: #f8efd4;
  --red: #9b2c2c;
  --red-bg: #f8e4e4;
  --shadow: 0 1px 2px rgba(16, 28, 48, 0.04), 0 12px 28px rgba(16, 28, 48, 0.05);
  --radius: 10px;
}

* { box-sizing: border-box; }
html, body, #root { min-height: 100%; }
body {
  margin: 0;
  color: var(--ink);
  background:
    radial-gradient(900px 420px at 8% -8%, rgba(160, 122, 50, 0.08), transparent 46%),
    linear-gradient(180deg, #f8f5f0 0%, var(--paper-2) 100%);
  font-family: "IBM Plex Sans", "Segoe UI", sans-serif;
  font-size: 15px;
  line-height: 1.45;
}

h1, h2, h3, .serif { font-family: "Libre Baskerville", Georgia, serif; font-weight: 400; }

.app { max-width: 1180px; margin: 0 auto; padding: 22px 22px 72px; }
.app:has(.report-layout.with-doc) { max-width: 1480px; }

.report-layout {
  display: block;
}
.report-layout.with-doc {
  display: grid;
  grid-template-columns: minmax(320px, 1fr) minmax(360px, 1.05fr);
  gap: 18px;
  align-items: start;
}
.doc-pane { position: sticky; top: 16px; }
.doc-frame {
  width: 100%;
  height: min(78vh, 920px);
  border: 1px solid var(--line);
  background: #fff;
}
.doc-image-wrap {
  max-height: min(78vh, 920px);
  overflow: auto;
  border: 1px solid var(--line);
  background: #fff;
}
.doc-image { display: block; max-width: 100%; height: auto; }
.doc-fallback {
  border: 1px dashed var(--line);
  padding: 28px 16px;
  text-align: center;
  background: #faf6ee;
}
@media (max-width: 980px) {
  .report-layout.with-doc {
    grid-template-columns: 1fr;
  }
  .doc-pane { position: static; }
  .doc-frame { height: 55vh; }
}

.masthead {
  display: grid;
  grid-template-columns: auto minmax(220px, 1fr) auto;
  align-items: center;
  gap: 20px;
  margin-bottom: 28px;
  padding-bottom: 18px;
```

```
  border-bottom: 1px solid var(--line);
}
.brand {
  display: flex;
  gap: 12px;
  align-items: center;
  background: none;
  border: 0;
  padding: 0;
  cursor: pointer;
  text-align: left;
  color: inherit;
}
.seal {
  width: 40px;
  height: 40px;
  border-radius: 50%;
  border: 1px solid var(--gold);
  background: var(--navy);
  color: #f4efe6;
  display: grid;
  place-items: center;
  font-family: "Libre Baskerville", serif;
  font-size: 13px;
  letter-spacing: 0.06em;
}
.brand h1 { margin: 0; font-size: 22px; color: var(--navy); letter-spacing: -0.02em; }
.brand p { margin: 3px 0 0; color: var(--muted); font-size: 12px; }

.mast-search { position: relative; }
.search {
  display: flex;
  align-items: center;
  gap: 8px;
  background: var(--card);
  border: 1px solid var(--line-strong);
  border-radius: 999px;
  padding: 8px 12px;
  color: var(--muted);
  box-shadow: 0 1px 0 rgba(255,255,255,0.7) inset;
}
.search:focus-within {
  border-color: var(--navy);
  color: var(--navy);
}
.search input {
  flex: 1;
  border: 0;
  background: transparent;
  font: inherit;
  color: var(--ink);
  outline: none;
  min-width: 0;
}
.search input::placeholder { color: #8b93a0; }
.search input[type="search"]::-webkit-search-cancel-button { display: none; }
.search-clear {
  appearance: none;
  border: 0;
  background: transparent;
  color: var(--muted);
  padding: 0;
  display: grid;
  place-items: center;
  cursor: pointer;
}
.search-results {
  position: absolute;
  z-index: 20;
  top: calc(100% + 6px);
```

```
  left: 0;
  right: 0;
  background: var(--card);
  border: 1px solid var(--line);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  padding: 6px;
  display: grid;
  gap: 2px;
}
.search-results button {
  appearance: none;
  border: 0;
  background: transparent;
  text-align: left;
  padding: 9px 10px;
  border-radius: 8px;
  cursor: pointer;
  font: inherit;
  display: grid;
  gap: 2px;
}
.search-results button:hover { background: #f4efe6; }
.search-results strong { color: var(--navy); font-size: 14px; }
.search-results span { color: var(--muted); font-size: 12px; }

.mast-nav {
  display: flex;
  align-items: center;
  gap: 4px;
}
.mast-nav button {
  appearance: none;
  border: 0;
  background: transparent;
  color: var(--muted);
  font: 500 13px "IBM Plex Sans", sans-serif;
  padding: 7px 10px;
  border-radius: 999px;
  cursor: pointer;
}
.mast-nav button:hover { color: var(--navy); background: rgba(26, 47, 77, 0.05); }
.mast-nav button.on { color: var(--navy); background: rgba(26, 47, 77, 0.07); }
.status-dot {
  margin-left: 6px;
  font-size: 11px;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  color: var(--muted);
  border: 1px solid var(--line);
  padding: 4px 8px;
  border-radius: 999px;
}
.status-dot.live { color: var(--green); border-color: #c9ddd1; background: var(--green-bg); }

.page-head {
  display: flex;
  align-items: flex-end;
  justify-content: space-between;
  gap: 16px;
  margin-bottom: 16px;
}
.page-head h2 { margin: 2px 0 4px; font-size: 28px; color: var(--navy); }
.toolbar {
  display: flex;
  align-items: flex-end;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 16px;
}
```

```
.toolbar .search { flex: 1; max-width: 360px; }

.card {
  background: var(--card);
  border: 1px solid var(--line);
  box-shadow: var(--shadow);
  padding: 22px 24px;
  border-radius: var(--radius);
}
.card + .card { margin-top: 16px; }
.card.quiet { background: #fbf8f2; }
.card.flush { padding: 0; overflow: hidden; }
.card.case-hero h2 { margin: 4px 0 8px; font-size: 26px; color: var(--navy); }
.banner.error {
  color: var(--red);
  background: var(--red-bg);
  border: 1px solid #e8c4c4;
  border-radius: var(--radius);
  padding: 10px 14px;
  margin-bottom: 16px;
}
.crumb-bar { margin-bottom: 14px; }
.text-link {
  appearance: none;
  border: 0;
  background: none;
  padding: 0;
  font: 500 13px "IBM Plex Sans", sans-serif;
  color: var(--navy);
  cursor: pointer;
}
.text-link:hover { text-decoration: underline; }
.meta-line {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 8px;
  margin: 0 0 16px;
  color: var(--muted);
  font-size: 13px;
}
.meta-line .dot {
  width: 3px;
  height: 3px;
  border-radius: 50%;
  background: #c5bdae;
}
.empty, .empty-copy {
  color: var(--muted);
  text-align: center;
  padding: 36px 16px;
}
.empty { display: grid; justify-items: center; gap: 10px; }

.dropzone {
  border: 1px dashed #c8bead;
  background: #faf7f1;
  padding: 36px 20px;
  text-align: center;
  cursor: pointer;
  transition: border-color 0.15s, background 0.15s;
  border-radius: 8px;
}
.dropzone.compact {
  display: flex;
  align-items: center;
  gap: 14px;
  text-align: left;
  padding: 14px 16px;
}
```

```
.dropzone.compact strong { display: block; color: var(--navy); font-size: 14px; }
.dropzone.compact p { margin: 2px 0 0; font-size: 12px; }
.dropzone.active { border-color: var(--gold); background: #fff8e8; }
.dropzone h2 { margin: 10px 0 6px; color: var(--navy); font-size: 22px; }
.dropzone p { margin: 0; color: var(--muted); }
.formats { margin-top: 14px; font-size: 12px; color: var(--muted); }

.batch-list {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
  flex-direction: column;
  gap: 8px;
}
.batch-item {
  width: 100%;
  display: flex;
  align-items: center;
  gap: 12px;
  text-align: left;
  border: 1px solid var(--line);
  background: #faf6ee;
  padding: 10px 12px;
  cursor: pointer;
  font: inherit;
  color: inherit;
}
.batch-item:hover { border-color: var(--gold); }
.batch-item.active {
  border-color: var(--navy);
  background: #fff;
  box-shadow: inset 3px 0 0 var(--gold);
}
.batch-index {
  width: 28px;
  height: 28px;
  border-radius: 50%;
  background: var(--navy);
  color: #f4efe6;
  display: grid;
  place-items: center;
  font-size: 12px;
  font-weight: 700;
  flex-shrink: 0;
}
.batch-meta {
  flex: 1;
  min-width: 0;
  display: flex;
  flex-direction: column;
  gap: 2px;
}
.batch-meta strong {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.batch-pill {
  font-size: 11px;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  font-weight: 700;
  color: var(--muted);
  flex-shrink: 0;
}
.batch-pill.complete { color: var(--green); }
.batch-pill.failed { color: var(--red); }
.batch-pill.running, .batch-pill.queued, .batch-pill.uploading { color: var(--yellow); }
.batch-pill.risk-red { color: var(--red); }
```

```
.batch-pill.risk-yellow { color: var(--yellow); }
.batch-pill.risk-green { color: var(--green); }

.btn {
  appearance: none;
  border: 0;
  background: var(--navy);
  color: white;
  padding: 8px 14px;
  font: 600 13px "IBM Plex Sans", sans-serif;
  cursor: pointer;
  border-radius: 8px;
}
.btn:disabled { opacity: 0.5; cursor: not-allowed; }
.btn.secondary { background: transparent; color: var(--navy); border: 1px solid var(--line-strong);
    +-> }
.btn.secondary:hover { border-color: var(--navy); }
.btn.gold { background: var(--gold); color: var(--navy-deep); }
.row { display: flex; gap: 10px; flex-wrap: wrap; align-items: center; }

.progress-track { height: 8px; background: #e7dfd0; margin: 14px 0 8px; }
.progress-fill { height: 100%; background: var(--navy); transition: width 0.3s; }
.stage-list { display: grid; gap: 8px; margin: 16px 0 0; padding: 0; list-style: none; }
.stage-list li {
  display: flex;
  justify-content: space-between;
  padding: 8px 10px;
  background: #f6f1e7;
  font-size: 13px;
}
.stage-list li.on { background: #e8eef7; color: var(--navy); font-weight: 600; }
.stage-list li.done { color: var(--green); }

.risk {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 6px 10px;
  font-weight: 700;
  letter-spacing: 0.02em;
  border-radius: 999px;
  font-size: 13px;
}
.risk.green { background: var(--green-bg); color: var(--green); }
.risk.yellow { background: var(--yellow-bg); color: var(--yellow); }
.risk.red { background: var(--red-bg); color: var(--red); }

.tabs { display: flex; gap: 0; border-bottom: 1px solid var(--line); margin: 8px 0 16px; }
.tabs button {
  background: none;
  border: 0;
  border-bottom: 2px solid transparent;
  padding: 10px 14px;
  font: 600 13px "IBM Plex Sans", sans-serif;
  color: var(--muted);
  cursor: pointer;
}
.tabs button.active { color: var(--navy); border-bottom-color: var(--gold); }

.finding {
  border: 1px solid var(--line);
  padding: 14px 16px;
  margin-bottom: 12px;
  background: white;
  border-radius: 8px;
}
.finding header { display: flex; justify-content: space-between; gap: 12px; align-items: baseline; }
.finding h3 { margin: 0; font-size: 16px; color: var(--navy); }
.sev { font-size: 11px; text-transform: uppercase; letter-spacing: 0.06em; font-weight: 700; }
.sev.critical, .sev.high { color: var(--red); }
```

```
.sev.medium { color: var(--yellow); }
.sev.low, .sev.info { color: var(--muted); }
.why { color: var(--muted); font-style: italic; }
.ai-lead {
  font-size: 12px;
  color: var(--muted);
  border-left: 3px solid var(--gold);
  padding-left: 8px;
  margin: 8px 0;
}
.heatmap { max-width: 100%; border: 1px solid var(--line); margin-top: 10px; }
.conf {
  height: 6px;
  background: #eee8dc;
  margin-top: 10px;
}
.conf > span { display: block; height: 100%; background: var(--navy); }

table.data {
  width: 100%;
  border-collapse: collapse;
}
table.data th,
table.data td {
  text-align: left;
  padding: 12px 14px;
  border-bottom: 1px solid var(--line);
  vertical-align: top;
}
table.data th {
  font-size: 11px;
  letter-spacing: 0.08em;
  text-transform: uppercase;
  color: var(--muted);
  font-weight: 600;
  background: #faf7f1;
}
table.data tr:last-child td { border-bottom: 0; }
table.data.cases td { vertical-align: middle; }
table.data .muted { color: var(--muted); font-size: 13px; }
table.data .mono { font-variant-numeric: tabular-nums; font-size: 13px; }
table.data .nowrap { white-space: nowrap; }
table.data .num { text-align: right; font-variant-numeric: tabular-nums; color: var(--navy); }
table.data .matter-name { color: var(--navy); font-weight: 600; }
table.data tr.click-row { cursor: pointer; }
table.data tr.click-row:hover td { background: #faf6ee; }
table.data select {
  max-width: 180px;
  font: inherit;
  padding: 6px 8px;
  border: 1px solid var(--line);
  background: #fff;
  border-radius: 6px;
}

.grid-2 { display: grid; grid-template-columns: 1.4fr 0.8fr; gap: 16px; }
@media (max-width: 860px) {
  .grid-2 { grid-template-columns: 1fr; }
  .masthead {
    grid-template-columns: 1fr;
    align-items: stretch;
  }
  .toolbar { flex-wrap: wrap; }
  .toolbar .search { max-width: none; }
}

.disclaimer, .footnote { font-size: 12px; color: var(--muted); line-height: 1.5; }
.error { color: var(--red); background: var(--red-bg); padding: 10px 12px; }
.actions { display: flex; gap: 8px; flex-wrap: wrap; margin: 12px 0 0; }
.kicker { font-size: 11px; letter-spacing: 0.14em; text-transform: uppercase; color: var(--gold);
```

```
    +-> font-weight: 600; margin: 0; }

.case-form {
  display: grid;
  gap: 12px;
  margin-top: 16px;
  max-width: 560px;
}
.case-form.inline {
  max-width: none;
  grid-template-columns: 1.4fr 1fr 0.7fr auto;
  align-items: end;
}
@media (max-width: 860px) {
  .case-form.inline { grid-template-columns: 1fr; }
}
.case-form label {
  display: grid;
  gap: 6px;
  font-size: 12px;
  color: var(--muted);
}
.case-form input {
  border: 1px solid var(--line);
  background: #fff;
  padding: 9px 11px;
  font: inherit;
  color: var(--ink);
  border-radius: 8px;
}
.linkish {
  background: none;
  border: none;
  padding: 0;
  font: inherit;
  color: var(--navy);
  text-decoration: underline;
  cursor: pointer;
  text-align: left;
}
.share-row {
  display: flex;
  gap: 10px;
  align-items: flex-start;
  margin: 16px 0;
  font-size: 14px;
}

.risk-summary {
  display: grid;
  gap: 10px;
}
.risk-drop {
  border: 1px solid var(--line);
  background: #fffcf7;
  padding: 0;
  border-radius: 8px;
}
.risk-drop > summary {
  list-style: none;
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  padding: 12px 14px;
  user-select: none;
}
.risk-drop > summary::-webkit-details-marker { display: none; }
.risk-drop[open] > summary { border-bottom: 1px solid var(--line); }
```

```
.risk-chip {
  font-size: 13px;
  font-weight: 600;
  padding: 4px 10px;
  border: 1px solid var(--line);
  border-radius: 999px;
}
.risk-chip.red { color: var(--red); background: var(--red-bg); border-color: #e2b4b4; }
.risk-chip.yellow { color: var(--yellow); background: var(--yellow-bg); border-color: #e2d4a4; }
.risk-chip.green { color: var(--green); background: var(--green-bg); border-color: #b7d4c2; }
.risk-count {
  min-width: 2rem;
  text-align: center;
  font-family: "Libre Baskerville", Georgia, serif;
  font-size: 20px;
  color: var(--navy);
}
.risk-links {
  list-style: none;
  margin: 0;
  padding: 8px 14px 12px;
  display: grid;
  gap: 8px;
}
.risk-links li {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 2px;
}

.toolbar.tight {
  align-items: center;
  margin-bottom: 10px;
}
.row-actions {
  display: flex;
  justify-content: flex-end;
  gap: 6px;
  white-space: nowrap;
}
.icon-btn {
  appearance: none;
  border: 1px solid transparent;
  background: transparent;
  color: var(--muted);
  padding: 5px 7px;
  border-radius: 7px;
  cursor: pointer;
  display: inline-flex;
  align-items: center;
  gap: 5px;
  font: 500 12px "IBM Plex Sans", sans-serif;
}
.icon-btn:hover:not(:disabled) {
  color: var(--navy);
  background: #f4efe6;
  border-color: var(--line);
}
.icon-btn.danger:hover:not(:disabled),
.btn.secondary.danger:hover:not(:disabled) {
  color: var(--red);
  background: var(--red-bg);
  border-color: #e2b4b4;
}
.btn.secondary.danger { color: var(--muted); }
.btn.secondary.danger { display: inline-flex; align-items: center; gap: 6px; }
.btn.secondary { display: inline-flex; align-items: center; gap: 6px; }
table.data th.actions-col,
table.data td.row-actions { width: 1%; text-align: right; }
```

```
.archive-panel {
  margin-top: 16px;
  background: var(--card);
  border: 1px solid var(--line);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
}
.archive-panel > summary {
  list-style: none;
  cursor: pointer;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 14px 18px;
  color: var(--navy);
  font-weight: 600;
}
.archive-panel > summary::-webkit-details-marker { display: none; }
.archive-panel > summary span { color: var(--muted); font-weight: 500; font-size: 13px; }

.case-chat.on-report .chat-thread {
  max-height: 240px;
}
.case-chat .chat-thread {
  max-height: 340px;
  overflow: auto;
  display: flex;
  flex-direction: column;
  gap: 10px;
  padding: 4px 2px 8px;
}
.chat-empty {
  color: var(--muted);
  font-size: 13px;
  line-height: 1.5;
}
.chat-empty p { margin: 8px 0 12px; }
.chat-hints {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}
.chat-hint,
.chat-cite {
  appearance: none;
  border: 1px solid var(--line-strong);
  background: #fbf8f2;
  color: var(--navy);
  border-radius: 999px;
  padding: 6px 10px;
  font: 500 12px "IBM Plex Sans", sans-serif;
  cursor: pointer;
  text-align: left;
}
.chat-hint:hover,
.chat-cite:hover { border-color: var(--navy); background: #fff; }
.chat-bubble {
  max-width: 92%;
  padding: 10px 12px;
  border-radius: 12px;
  font-size: 14px;
  line-height: 1.5;
  white-space: pre-wrap;
}
.chat-bubble p { margin: 0; }
.chat-bubble.user {
  align-self: flex-end;
  background: var(--navy);
  color: #f4efe6;
}
```

```
.chat-bubble.assistant {
  align-self: flex-start;
  background: #f4efe6;
  border: 1px solid var(--line);
  color: var(--ink);
}
.chat-cites {
  display: flex;
  flex-wrap: wrap;
  gap: 6px;
  margin-top: 10px;
}
.chat-inline-link {
  appearance: none;
  border: 0;
  background: none;
  padding: 0;
  color: var(--navy);
  font: inherit;
  font-weight: 600;
  text-decoration: underline;
  text-underline-offset: 2px;
  cursor: pointer;
}
.chat-inline-link:hover { color: var(--gold); }
.chat-compose {
  display: grid;
  grid-template-columns: 1fr auto;
  gap: 8px;
  align-items: end;
  margin-top: 8px;
}
.chat-compose textarea {
  width: 100%;
  resize: vertical;
  min-height: 52px;
  border: 1px solid var(--line-strong);
  border-radius: 10px;
  padding: 8px 10px;
  font: inherit;
  color: var(--ink);
  background: #fff;
}
.chat-compose textarea:focus {
  outline: none;
  border-color: var(--navy);
}
.gotcha-board .gotcha-item {
  border: 1px solid var(--line);
  border-radius: 10px;
  padding: 12px 14px;
  margin-top: 10px;
  background: #fff;
}
.gotcha-board .gotcha-item h3 {
  margin: 0 0 8px;
  font-size: 16px;
  color: var(--navy);
}
.gotcha-quote {
  margin: 6px 0;
  font-size: 13px;
  line-height: 1.45;
}
.gotcha-quote strong { color: var(--navy); }
.gotcha-why {
  margin: 8px 0 0;
  font-size: 13px;
  line-height: 1.45;
  color: var(--ink);
```

```
}
.finding.gotcha-card {
  border-color: #c9a227;
  background: #fffdf6;
}
```